RESEARCH FINDINGS / NOVEMBER 2026

Finding the first move in a forced mate

Runtime, accuracy, and failure modes of Minimax and Monte Carlo tree search

Akbar Aman · Prepared for La Min Aung, group poster coordinator
November 3, 2026 · Group findings due November 5

My objective and our agreed questions

Following our October 31 discussion and Min's email, I investigated how Minimax and MCTS compare in execution time, which produces more accurate moves, and how depth, position complexity, and computational limits affect incorrect moves. I used the 20 supplied mate-in-three problems to develop findings that we can compare with everyone's results for our poster.

October setting	Calls	Finished	Finished key hits	Median time*
Minimax, 3 plies	20	20	6	0.535 s
Minimax, 5 plies	20	2	2	8.195 s
MCTS, 1,000 simulations	60	60	9	2.724 s

*Among finished calls. MCTS used three seeds on each of the same 20 positions. The five-ply median describes only puzzles 16 and 18.

What I tested and found

I recorded six correct first moves from three-ply Minimax and nine from MCTS across five distinct positions. In puzzle 6, all three MCTS seeds chose to take a rook with the queen. Each missed a queen check that forces mate. I examine this choice through the board position, a defensive reply, and the recorded root visits.

Five plies reach the attacker's third move. My full-depth search finished on two puzzles and found both keys. I stopped the other 18 calls at 20 seconds. Eight partial answers matched the key, but I cannot infer how much work remained.

In a separate November run, I gave both algorithms the same time ceilings. Increasing the ceiling from a quarter second to one second increased the returned keys from 4 to 5 across 20 Minimax decisions and from 3 to 6 across 60 MCTS decisions. I interpret these observations within my puzzle set, implementations, machine, and run order.

My repository, original reports, measured results, and poster figures

01 / STUDY DESIGN

How I checked and scored the moves

From course diagrams to checkable positions

In September, I measured runtime on eight deliberately varied boards without a known optimal move for each. In October, I used the corrected course sheet, *Positions2.pdf*, for a specific mate-finding objective. I checked all 1,280 squares against the diagram glyphs. Position 3 has Black to play. The others have White to play. I recorded the indicated side, no castling or en passant rights, and clocks 0 and 1 in each FEN. For positions 1 and 4, I also checked three alternative Black castling-rights settings and found the same unique key in each.

A *ply* is one move by one player. The attacker moves on plies 1, 3, and 5. My exact verifier needs one winning continuation at attacker nodes, while every legal defensive reply must still lose within the horizon. I tested every legal first move at bounds 1, 3, and 5. I found no shorter forced mate and exactly one five-ply key per position. I then replayed a saved winning strategy against every legal defense with a separate certificate checker. Both checks use the same chess rules library as my search programs.

In November, I replayed every printed main line. All 20 keys agreed with my verifier, and all 20 lines ended in checkmate. Thirteen printed lines differed from my stored sample. A sample shows one path through a strategy, while my certificate covers the defender's alternatives.

Two ways of deciding

My plain Minimax implementation searches every legal move to its depth, maximizing for White and minimizing for Black. I give checkmate priority over material. At a nonterminal leaf, I count pawns, minor pieces, rooks, and queens as 1, 3, 5, and 9. I retain the first legal move when evaluations tie. On timeout, my code keeps the best fully evaluated root branch, or the first legal move if none finished.

In MCTS, I expand a random untried move, use UCT to revisit branches, and simulate random legal play for up to 40 plies. I use win, loss, or draw rewards at terminal outcomes and a bounded material reward at the cutoff. My code returns the most visited root move, breaking ties by the mover's reward. A favorable sample can occur even when the defender has a reply that defeats the puzzle objective.

What counts as accuracy

I count a hit when the returned move equals the unique verified key. I check the full continuation separately. In my October analysis, I distinguish completed key hits from keys returned on timeout. In November, I score the move available when time runs out.

02 / EXECUTION TIME AND ACCURACY

Search effort depends on the position

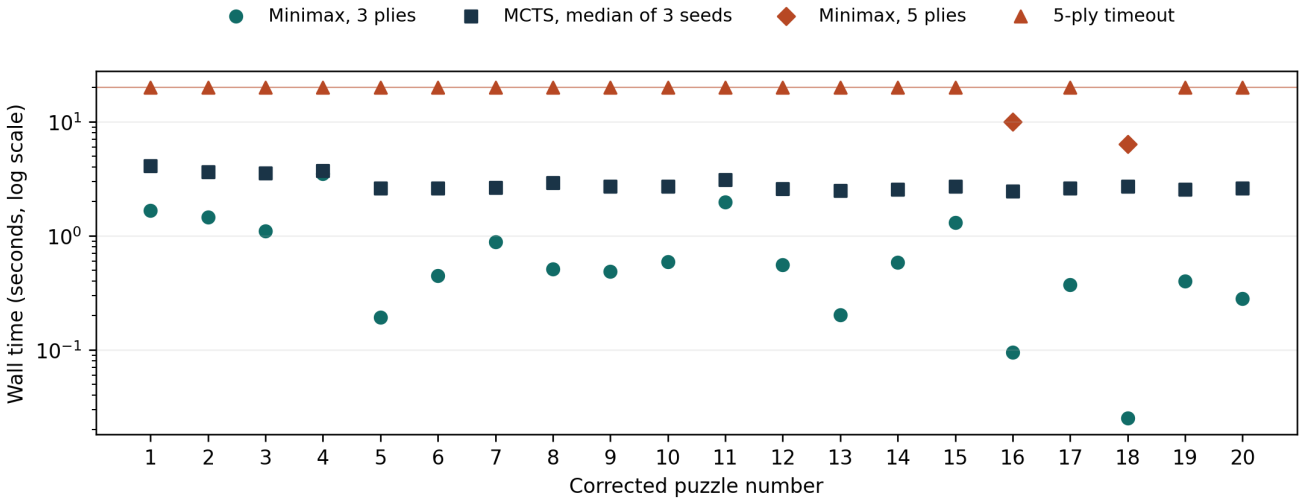


Figure 1. October wall time by puzzle. Each square gives the median across three MCTS seeds. Triangles mark the 20-second ceiling. The eventual full-depth times for those positions are censored.

RQ1. Execution time

My three-ply Minimax runs finished all 20 searches. I measured medians of 2.724 s for the 60 MCTS calls and 0.535 s for Minimax under their different requested workloads. My earlier eight-board study also showed why piece count alone is inadequate. At depth three, two 16-piece boards had 15 versus 53 legal first moves, visited 2,978 versus 123,342 nodes, and took 0.087 versus 3.586 s at the median.

RQ2. Move accuracy

I recorded completed-call hit rates of 30% for three-ply Minimax and 15% for MCTS. Each MCTS seed found three keys, with five distinct puzzles hit across the seeds. I treat the 60 trials as repeated observations on 20 boards.

Shared time limits, measured November 3

I shuffled all 160 calls with order seed 496. I requested three plies for Minimax and 100,000 iterations for MCTS so its clock would determine the sample, using seeds 0, 1, and 2. I allowed either method to return early and counted returned decisions, including partial calls.

Setting	Returned hits	Finished	Hits when finished	Median s	Median sims
0.25 s Minimax	4/20	4	1	0.2501	–
0.25 s MCTS	3/60	0	0	0.2502	90
1.00 s Minimax	5/20	15	5	0.5286	–
1.00 s MCTS	6/60	0	0	1.0002	341

I used a shared ceiling to give each algorithm the same opportunity to spend time. Actual time used can differ. My cooperative clock checks can overshoot slightly, and timeout decisions can change on replay.

03 / INCORRECT MOVES

A defense explains what a miss costs

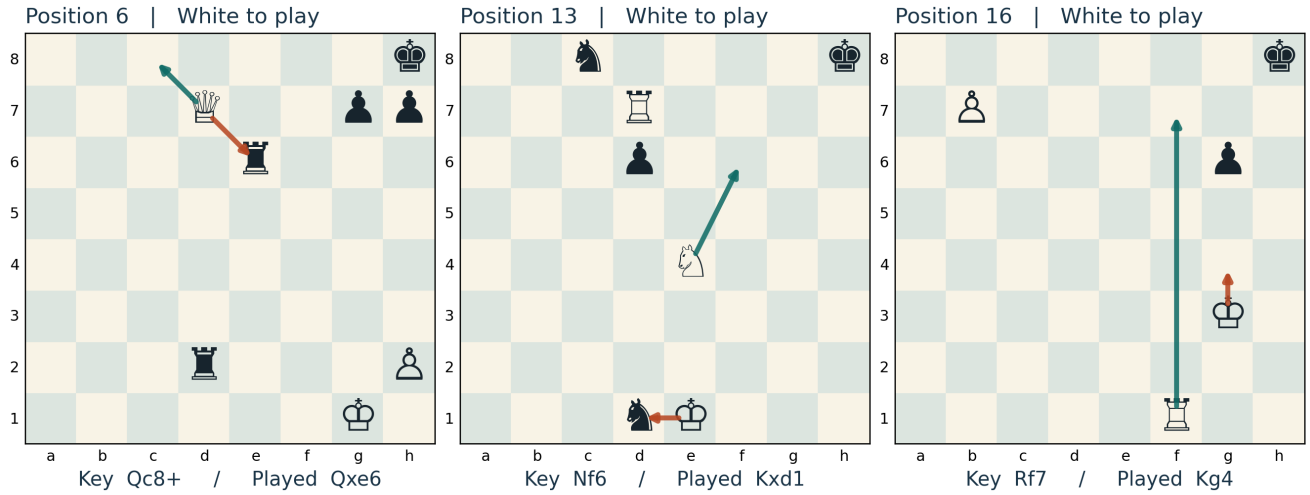


Figure 2. Teal arrows show the verified keys. Orange arrows show moves actually returned in October. All three boards have White to play.

6. A rook capture misses the back-rank mate

I recorded **Qxe6** for all three MCTS seeds and verified that **...Rd8** prevents White from forcing mate within the remaining three plies. The key is **Qc8+**. Against ...Rd8, White has Qxd8+ Re8 Qxe8 mate. At seed 0, I recorded 88 visits to the capture and 56 to the key, with mean White rewards of 0.544 and 0.443. Of 1,000 rollouts, 881 reached the material cutoff. I can explain the final visit-based selection from these records, but I have not isolated the effect of changing the rollout policy.

13. Taking a knight spends the key tempo

My three-ply Minimax run and all three MCTS seeds played **Kxd1**. I verified that **...Kg8** escapes the prescribed mate horizon. White has **Nf6**, with the printed continuation ...Ne7, Rxe7 Ne3, Rh7 mate. My five-ply Minimax call returned Nf6 at timeout, before it finished evaluating the root.

16. Looking two plies further changes the decision

My three-ply Minimax run chose **Kg4**, after which **...Kg8** prevents mate within three more plies. My five-ply run finished in 10.012 s and chose **Rf7**. After ...Kg8, Rd7 g5, b8=Q is mate. By changing depth with the same evaluator, I exposed the mate and changed the first move. In position 18, both depths finished and selected the same key.

A correction to the solution commentary

For position 18, the booklet gives Rxg7 Kf8 Ne6+ Ke8 c8=R mate and says queen promotion stalemates. I replayed this position and found that **both c8=R and c8=Q give checkmate**, while bishop and knight promotion each give stalemate. This does not change my first-move accuracy counts.

04 / INTERPRETATION AND REUSE

What the poster can conclude

Evidence for the third research question

I observed a depth effect in puzzle 16, where my completed five-ply search changed a wrong move into the verified key. I also observed different search costs. Position 18 has three legal first moves and took 0.025 s at depth three, while position 4 has 43 and took 3.513 s. I use root mobility to describe one layer, alongside deeper branching, checks, and terminal positions. I have not isolated its independent effect on accuracy.

When I increased the November ceiling, median MCTS simulations rose from 90 to 341 and more returned moves matched the key. In my October logs, 89.56% of 60,000 rollouts ended at the evaluation cutoff. I consider the material heuristic and random defense sampling plausible contributors to missed tactics. I did not vary those mechanisms independently.

Limits and attribution

I tested twenty selected tactical compositions with unique keys, nineteen with White to move. I interpret my results within that scope. Five-ply completion was highly selective. I used three seeds per MCTS condition and one November run per seed and ceiling. Machine load affects timeouts. My exact solver uses separate search logic but shares the rules engine, so it cannot independently detect a defect common to that engine.

I prepared this contribution for our UIC MURL Chess poster, which Min coordinates. At our October 31 meeting, we agreed to investigate all three questions and pool our results. Min's email set November 5 for the findings. I have retained the settings and provenance of my runs so we can compare them with other members' experiments. I used the Spring 2026 poster as a layout reference.

Using these findings in the poster

I suggest puzzle 6 for the clearest wrong-move comparison and puzzle 16 for the depth example. My runtime chart and per-puzzle table address the first two questions. We can select these alongside everyone's findings. I have also prepared a concise Google Doc and PDF snapshot, with my original reports and reproducible evidence linked from the repository.

Reproduction and sources

I have included my code, FENs, all 100 October search calls, 60 verifier calls, strategy certificates, and 160 November decisions in the repository. My environment records identify Windows 11 build 26200, AMD64 Family 25 Model 80, CPython 3.12.14, and chess 1.11.2. I have placed my original PDFs in `Reports/` and archived the corrected diagrams and solution booklet with their provenance.

Algorithm background: Russell and Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., ch. 5; Kocsis and Szepesvári, Bandit Based Monte-Carlo Planning (2006). The repository bibliography and source code identify the implementation choices. Numerical claims above come from the course benchmark logs.